# Technical Requirement Document (TRD)

# Project Name

DevConnect

Professional Networking Platform for Developers

---

# 1. System Architecture Overview

## Architecture Style

Modular Monolithic Architecture

Reason:

- Faster development for MVP
- Easier deployment
- Lower operational complexity
- Can later evolve into microservices if growth demands

## High-Level Architecture

Frontend (React SPA) ↓ REST APIs + WebSockets ↓ Express.js Backend ↓ Prisma ORM ↓ PostgreSQL (Supabase) ↓ Redis (Caching + Socket Adapter)

---

## Backend Modules

Auth Module

Profile Module

Skill Module

Project Module

Connection Module

Feed Module

Recommendation Module

Messaging Module

Notification Module

Search Module

Each module owns:

- Routes
- Controllers
- Services
- Validation
- Database Access

---

# 2. Frontend Responsibilities

Frontend should remain presentation-focused.

## Authentication

- Register
- Login
- Logout
- Token storage
- Protected routes

---

## Profile Management

- Create profile
- Edit profile
- Upload avatar
- Manage skills
- Manage projects

---

## Networking

- Search developers
- View profiles
- Send requests

- Accept requests
- Reject requests

---

## Feed

- Create posts
- View feed
- Like posts
- Comment on posts

---

## Recommendations

- Display recommended users
- Show recommendation reason

---

## Messaging

- Real-time chat UI
- Chat history
- Notification indicators

---

## Frontend Should NOT

- Implement business logic
- Calculate recommendations
- Verify permissions
- Directly access database

All business logic must remain in backend.

---

# 3. Backend Responsibilities

Backend is the source of truth.

## Authentication

Responsibilities:

- Register users

- Login users
- Hash passwords
- Issue JWT tokens
- Verify tokens
- Refresh tokens

---

## Authorization

Validate:

- Resource ownership
- Connection permissions
- Messaging permissions

---

## Profile Management

- CRUD Profiles
- CRUD Projects
- Skill Management

---

## Social Graph

- Connection requests
- Accept requests
- Reject requests
- Retrieve connections

---

## Feed Engine

- Create post
- Delete post
- Like post
- Comment post
- Feed aggregation

---

### Recommendation Engine

Generate recommendations using:

- Shared skills
- Mutual connections

---

### Messaging

- Persist messages
- Deliver messages through Socket.IO
- Track read status

---

### Search

- User search
- Skill search

---

## 4. Database Schema Proposal

### Users

```
id UUID PK
email VARCHAR UNIQUE
password_hash TEXT
full_name VARCHAR
headline VARCHAR
bio TEXT
avatar_url TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

---

### Skills

```
id SERIAL PK
name VARCHAR UNIQUE
```

---

## UserSkills

```
user_id UUID FK
skill_id INT FK

PRIMARY KEY(user_id, skill_id)
```

---

## Projects

```
id UUID PK
user_id UUID FK
title VARCHAR
description TEXT
repo_url TEXT
project_url TEXT
created_at TIMESTAMP
```

---

## Connections

```
id UUID PK
sender_id UUID FK
receiver_id UUID FK
status ENUM
created_at TIMESTAMP
updated_at TIMESTAMP
```

Status:

pending

accepted

rejected

---

## Posts

```
id UUID PK
user_id UUID FK
content TEXT
media_url TEXT
created_at TIMESTAMP
updated_at TIMESTAMP
```

---

## Likes

```
user_id UUID FK
post_id UUID FK

PRIMARY KEY(user_id, post_id)
```

---

## Comments

```
id UUID PK
user_id UUID FK
post_id UUID FK
content TEXT
created_at TIMESTAMP
```

---

## Messages

```
id UUID PK
sender_id UUID FK
receiver_id UUID FK
message_text TEXT
is_read BOOLEAN
created_at TIMESTAMP
```

---

### Notifications

```
id UUID PK
user_id UUID FK
type VARCHAR
reference_id UUID
is_read BOOLEAN
created_at TIMESTAMP
```

---

### RefreshTokens

```
id UUID PK
user_id UUID FK
token_hash TEXT
expires_at TIMESTAMP
created_at TIMESTAMP
```

---

# 5. API Structure

Base URL

```
/api/v1
```

---

### Auth APIs

```
POST /auth/register
POST /auth/login
POST /auth/refresh
POST /auth/logout
GET  /auth/me
```

---

## Profile APIs

```
GET    /users/:id
PUT    /users/profile
GET    /users/search
```

---

## Skills APIs

```
GET    /skills
POST   /skills
DELETE /skills/:id
```

---

## Projects APIs

```
POST   /projects
GET    /projects/:id
PUT    /projects/:id
DELETE /projects/:id
```

---

## Connection APIs

```
POST /connections/request
POST /connections/accept
POST /connections/reject

GET  /connections
GET  /connections/pending
```

---

## Feed APIs

```
POST   /posts
GET    /posts/feed
DELETE /posts/:id
```

### Like APIs

```
POST /posts/:id/like
```

---

### Comment APIs

```
POST /posts/:id/comment
GET  /posts/:id/comments
```

---

### Recommendation APIs

```
GET /recommendations
```

---

### Messaging APIs

```
GET  /messages/:userId
POST /messages
```

---

### Notification APIs

```
GET  /notifications
PUT  /notifications/:id/read
```

---

## 6. Authentication Strategy

### Access Token

JWT

Contains:

```
{
  "userId": "...",
  "email": "...",
  "role": "user"
}
```

Expiration:

15 minutes

---

## Refresh Token

Stored:

- Database
- HTTP-only Cookie

Expiration:

30 days

---

## Password Storage

bcrypt

Rounds:

12

---

## Protected APIs

Use:

```
Authorization: Bearer <token>
```

Middleware verifies:

- Signature
- Expiration
- User existence

# 7. Third-Party Dependencies

## Core Backend

Express.js

Prisma

PostgreSQL

---

## Authentication

jsonwebtoken

bcrypt

---

## Validation

Zod

---

## File Upload

Multer

Cloudinary

---

## Realtime

Socket.IO

---

## Caching

Redis

ioredis

---

### Logging

Winston

Morgan

---

### Security

Helmet

CORS

Rate Limiter

---

### Testing

Jest

Supertest

---

## 8. Scalability Considerations

### Database

Indexes:

```
users(email)

skills(name)

connections(sender_id, receiver_id)

connections(status)

posts(created_at DESC)

messages(sender_id, receiver_id)
```

---

## Feed Performance

Initial MVP:

Pull-based feed generation

Avoid pre-computed feeds.

---

## Caching

Redis cache:

- Feed responses
- Recommendations
- User profiles

TTL:

5–15 minutes

---

## File Storage

Never store images in database.

Store only URLs.

---

## Socket Scaling

Use Redis Adapter for Socket.IO.

Allows horizontal scaling later.

---

## Deployment Strategy

Frontend:

- Vercel

Backend:

- Render/Railway

Database:

- Supabase PostgreSQL

Cache:

- Redis Cloud

---

# 9. Non-Functional Requirements

Availability: 99%+

---

API Response Time:

P95 < 300ms

---

Security:

- JWT Authentication
- Input Validation
- Rate Limiting
- Password Hashing
- Secure Cookies

---

Code Quality:

- ESLint
- Prettier
- Modular Architecture
- Service Layer Pattern

---

# Final Technical Recommendation

Keep DevConnect as a modular monolith using:

React + Express + Prisma + PostgreSQL + Redis + Socket.IO

This architecture comfortably supports thousands of users while remaining simple enough for a student team to build, maintain, deploy, and explain during internships and placements.